OHS REMOTE — CAPSTONE DOCUMENTATION

Capstone347 • CSD-235

Supporting Documentation

Architecture, ERD, API Overview, and Submission Index

Capstone347 — CSD-235

April 14, 2026

Contents

1 Purpose of this document **2**

2 Entity Relationship Diagram **2**
2.1 Key tables at a glance 3

3 High-level architecture **3**
3.1 External systems the backend talks to 4

4 Happy-path order sequence **5**

5 API overview **6**
5.1 Public: order creation flow 6
5.2 End-user authentication and dashboard 7
5.3 Admin 7
5.4 SJP (Safe Job Procedure) generation 8

6 Screenshot appendix **8**

7 Submission index **8**

1 Purpose of this document

This is the consolidated "additional materials" package for the OHS Remote capstone submission. It exists so that a reader who has already gone through the User Guide and the two Developer Setup Guides has a single place to find:

- the database Entity Relationship Diagram (ERD),
- the high-level system architecture,
- the happy-path order sequence across the backend and its external systems,
- a narrative map of every HTTP endpoint the backend exposes, and
- the submission index — links to the repositories, the GitHub Project board, and the live Swagger UI.

For exact request and response schemas, the authoritative source is always the live Swagger UI at `http://localhost:8000/docs` (generated from the Pydantic models).

2 Entity Relationship Diagram

The diagram below is exported from the project's `docs/erd.drawio` source. Core aggregates are **Order**, **Plan**, **Company**, **Document**, and the supporting entities for authentication, admin staff, and SJP (Safe Job Procedure) generation jobs.

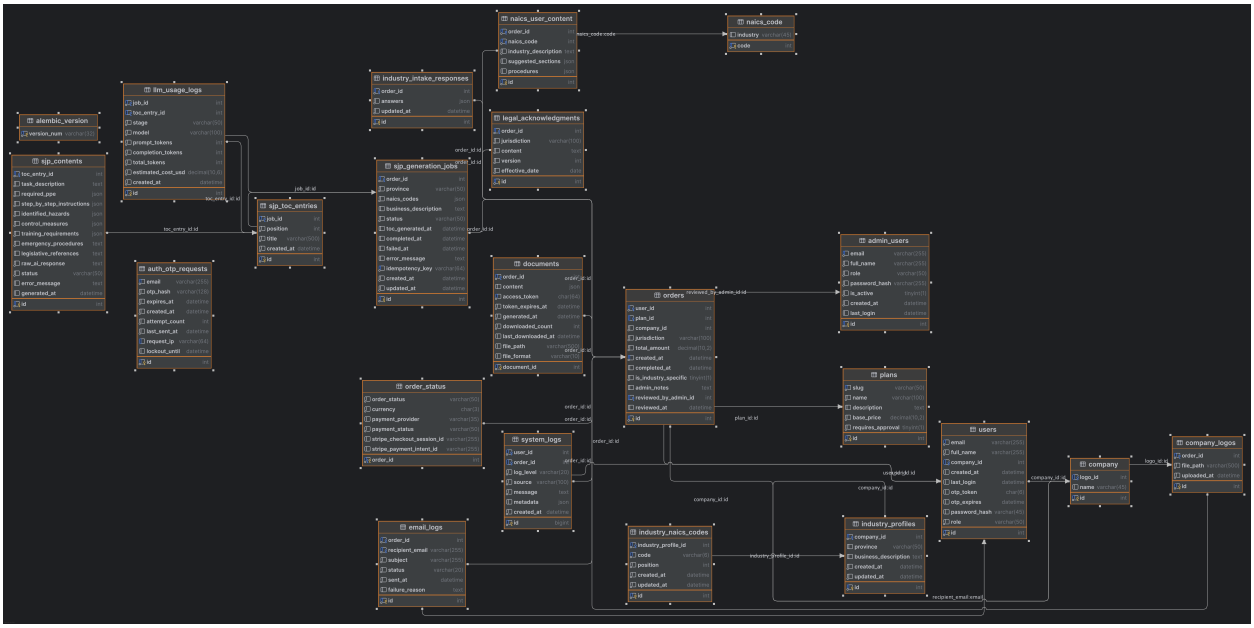

Figure 1: OHS Remote database ERD. The `orders` table is the central aggregate — it references a `plans` row, owns a `companies` row, owns one or more `documents` rows, and is linked to `order_status_history` entries that record every status transition during the order lifecycle.

2.1 Key tables at a glance

- **plans** — catalog of purchasable packages (Basic, Comprehensive, Industry-Specific SJP). Price, slug, and the template identifier live here.
- **orders** — the central aggregate. Tracks the full lifecycle (`draft` → `payment_initiated` → `paid` → `processing` → `completed`). Holds the customer's email, chosen jurisdiction, total amount, and Stripe session identifiers.
- **companies** — company details captured during the order flow: legal name, address, logo path, NAICS codes, industry intake answers.
- **documents** — generated deliverables. Stores the rendered file path, a token-gated download URL, the token expiry, and the associated order.
- **order_status_history** — append-only audit log of every status change on an order.
- **users** — end-customer accounts, created lazily after a successful payment so the customer can log in and re-download their documents later.
- **otp_codes** — one-time codes issued by the email OTP login flow, with expiry and rate-limit metadata.
- **admin_staff** — separate admin accounts (distinct from end users), with role-based access to the admin API.
- **sjp_generation_jobs** — async LLM-powered SJP generation jobs, with phase, progress, and result pointers.
- **llm_usage_logs** — per-call LLM cost accounting (tokens in/out, model, job reference) for monitoring OpenAI spend.

3 High-level architecture

The backend follows a strict four-layer architecture: **API** (FastAPI routers) calls **Services** (business logic), which call **Repositories** (SQLAlchemy data access), which own **Models** (ORM definitions). Routes never touch repositories directly. Services never import from the API layer.

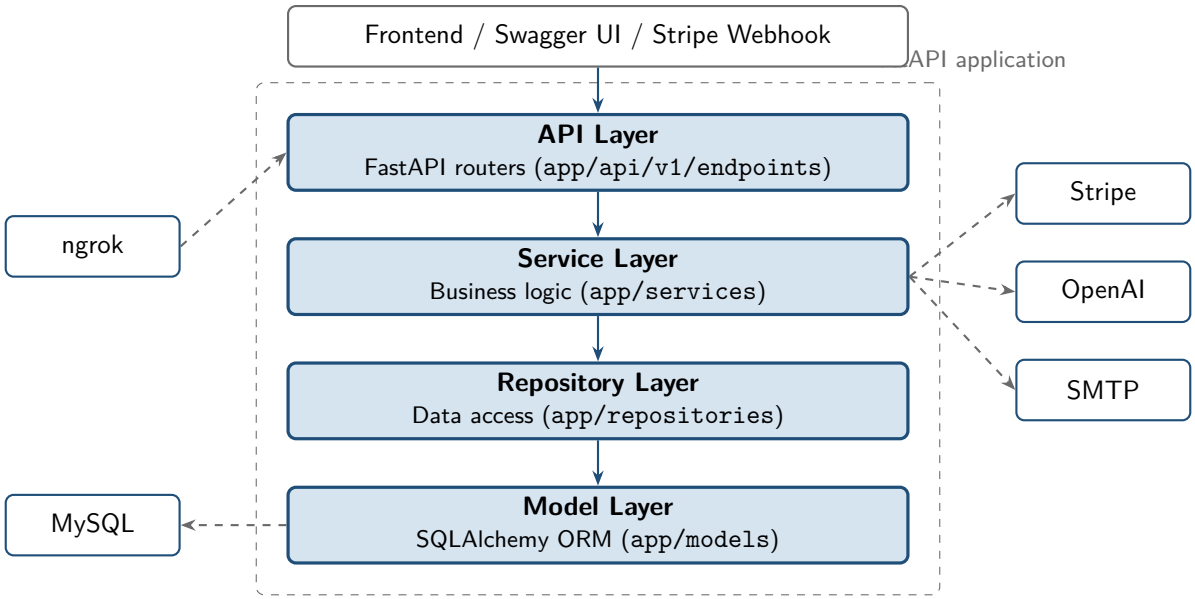

Figure 2: High-level architecture. The FastAPI application is layered (API → Service → Repository → Model) and is bounded by external systems: MySQL for persistence, Stripe for payments, OpenAI for SJP generation, SMTP (Gmail) for email delivery, and ngrok for exposing the webhook endpoint to Stripe during development.

3.1 External systems the backend talks to

System	Purpose	Where it is used
MySQL 8.0	Persistent storage for orders, users, documents, admin data, and SJP jobs.	`app/repositories/*` via SQLAlchemy. Schema versioned with Alembic.
Stripe	Payment processing via Checkout Sessions. Webhooks drive order status transitions.	`app/services/stripe_provider.py`, `app/api/v1/endpoints/payments.py`, `webhooks.py`.
OpenAI	LLM-powered SJP generation (content + table of contents).	`app/services/llm_provider.py`, `sjp_generation_service.py`.
SMTP (Gmail)	Transactional email: OTP codes, document delivery, admin notifications.	`app/services/email_*.py`, Jinja2 templates in `templates/emails/`.
ngrok	Tunnels the local webhook endpoint to a public HTTPS URL so Stripe can reach it during development.	`docker-compose.yml` (`ngrok` service), used only in the dev stack.

4 Happy-path order sequence

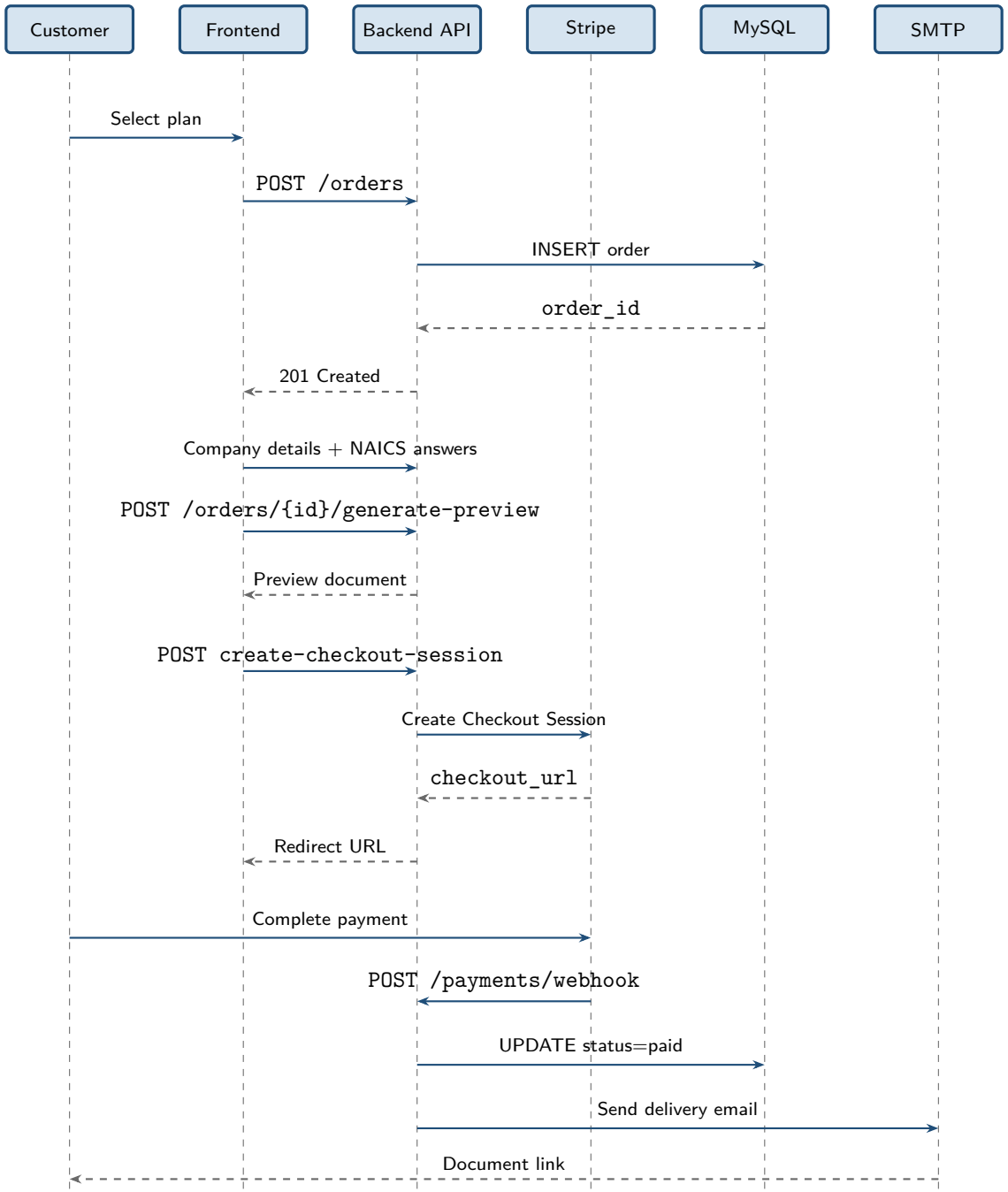

Figure 3: End-to-end customer order sequence. Solid arrows are outgoing calls; dashed arrows are responses. Stripe drives the final status transition by calling back into the webhook endpoint, which writes the paid status to MySQL and fires the delivery email.

In words:

1. The customer selects a plan (`GET /plans`) and creates an order (`POST /orders`). The returned

`order_id` threads through every subsequent call.

2. The frontend submits company details and an optional logo upload via `PATCH /orders/{id}/company-details`.
3. The frontend fetches industry intake questions from `GET /industry/intake-questions`, then submits answers via `PUT /industry/{order_id}/intake-answers`.
4. A document preview is generated with `POST /orders/{order_id}/generate-preview`, and the customer views it via `GET /documents/{document_id}/preview`.
5. The customer reads the legal disclaimer at `GET /legal-disclaimers/...`, acknowledges the terms, and a Stripe Checkout session is created via `POST /payments/orders/{order_id}/create-checkout-session`.
6. The customer is redirected to Stripe, completes payment, and Stripe calls back to `POST /api/v1/payments/webhook`. The webhook handler marks the order paid, generates the final document, and emails it to the customer.
7. The customer is redirected back to a success page on the frontend. A `users` row is created if one does not yet exist, and the customer can log in with email OTP to re-download later.

5 API overview

The backend exposes its HTTP interface under the base path `/api/v1`. All endpoints return JSON. The unauthenticated parts of the order flow (plans, order creation, company details, preview, legal, checkout session) are public by design, matching the product requirement that a customer can complete a purchase without creating an account first. Accounts and the dashboard appear after payment.

Authenticated end-user endpoints require the `auth_session` httpOnly cookie, which is set by the OTP verification endpoint. Admin endpoints use a separate auth flow (`POST /admin/auth/login`) and return a distinct session token.

5.1 Public: order creation flow

Endpoint	Auth	Purpose
`GET /plans`	None	List purchasable plans.
`POST /orders`	None	Create a new order, returning `order_id`.
`PATCH /orders/{id}/company-details`	None	Save company info and optional logo upload.
`GET /industry/intake-questions`	None	Fetch NAICS-driven intake questions.
`PUT /industry/{id}/intake-answers`	None	Persist intake answers.
`GET /industry/{id}/intake-answers`	None	Read back stored answers.
`POST /orders/{id}/generate-preview`	None	Generate a preview document.
`GET /documents/{id}/preview`	None	Stream the preview file.

Endpoint	Auth	Purpose
`GET /legal-disclaimers/{plan_id}/{jurisdiction}`	None	Fetch plain-text legal terms.
`POST /orders/{id}/acknowledge-terms`	None	Record terms acceptance.
`GET /payments/stripe/config`	None	Return the Stripe publishable key.
`POST /payments/orders/{id}/create-checkout-session`	None	Create a Stripe Checkout Session.
`POST /payments/webhook`	Stripe-signed	Stripe webhook handler — drives status transitions.

5.2 End-user authentication and dashboard

Endpoint	Auth	Purpose
`POST /auth/request-otp`	None	Email a one-time code. Rate-limited.
`POST /auth/verify-otp`	None	Verify the code and set the `auth_session` cookie.
`GET /auth/me`	Cookie	Return the current user.
`POST /auth/logout`	Cookie	Invalidate the session.
`GET /orders`	Cookie	List the current user's orders.
`GET /orders/{id}`	Cookie	Order detail, including status history.
`GET /orders/{id}/documents`	Cookie	List generated documents for an order.
`GET /documents/{id}/download`	Token	Download via token-gated URL.
`GET /orders/{id}/download`	Token	Download via token-gated URL (order scope).
`POST /documents/{id}/refresh-token`	Cookie	Mint a new download token when the old one expires.

5.3 Admin

Endpoint	Auth	Purpose
`POST /admin/auth/login`	None	Admin login (email + password).
`GET /admin/auth/me`	Admin	Current admin profile.
`POST /admin/auth/logout`	Admin	Invalidate admin session.
`POST /admin/auth/change-password`	Admin	Change password.
`GET /admin/orders`	Admin	List all orders with filters.
`GET /admin/orders/pending-review`	Admin	Approval queue (FIFO).
`GET /admin/orders/{id}`	Admin	Full admin order detail.
`PATCH /admin/orders/{id}/notes`	Admin	Update internal notes.
`POST /admin/orders/{id}/resend-email`	Admin	Re-send delivery email.
`POST /admin/orders/{id}/regenerate-document`	Admin	Regenerate the deliverable.
`POST /admin/orders/{id}/approve`	Admin	Approve a review-pending order.
`GET /admin/customers`	Admin	List end customers.
`GET /admin/customers/{id}`	Admin	Customer detail.

Endpoint	Auth	Purpose
`GET /admin/plans`	Admin	List plans (admin view).
`PATCH /admin/plans/{id}/approval-setting`	Admin	Toggle approval requirement.
`GET /admin/templates`	Admin	List DOCX templates.
`POST /admin/templates/{slug}`	Admin	Upload a new template.
`GET /admin/templates/{slug}/download`	Admin	Download a template.
`GET /admin/stats/dashboard`	Admin	Dashboard stats (counts, revenue, etc.).
`GET /admin/logs/emails`	Admin	Email delivery log.
`GET /admin/staff`	Admin	List admin users.
`POST /admin/staff`	Admin	Create an admin user.

5.4 SJP (Safe Job Procedure) generation

SJP generation is an async LLM pipeline. A job is created, progresses through phases (TOC, content, document assembly), and can be polled for progress. See the full flow in `docs/SJP_FRONTEND_GUIDE.md` in the repository.

- `POST /sjp/orders/{id}/generate` — start an SJP generation job.
- `GET /sjp/jobs/{job_id}` — poll status and progress.
- `GET /sjp/jobs/{job_id}/result` — fetch the resulting document identifier once the job completes.

6 Screenshot appendix

Screenshots referenced elsewhere in the documentation set live under `documents/assets/screenshots/`, organized by audience: `customer/`, `dashboard/`, `admin/`, `developer/`, and `github_project/`. See `user_guide.tex` and `project_management.tex` for the concrete figures each one appears in.

7 Submission index

The complete set of artifacts for the CSD-235 final submission:

Item	Location
GitHub Organization	https://github.com/Capstone347
Backend Repository	https://github.com/Capstone347/ohs_backend
Frontend Repository	https://github.com/Capstone347/ohs_frontend
GitHub Project Board	https://github.com/orgs/Capstone347/projects/2
Live API Documentation	http://localhost:8000/docs (when the stack is running)
User Guide	`documents/user_guide.pdf`
Developer Setup Guide (BE)	`documents/developer_setup_backend.pdf`
Developer Setup Guide (FE)	`documents/developer_setup_frontend.pdf`
Project Management Narrative	`documents/project_management.pdf`

Item	Location
Supporting Documentation (this file)	`documents/supporting_docs.pdf`
Presentation Slide Deck	Delivered separately by the team.